SEMANA AI DATA ENGINEER · 2026

Dia 3
Agente

De “eu decido” para “o agente decide”

Instrutor: Luan Moreno
AIDE BRASIL · aide-brasil.ai

AGENTE LANGCHAIN CHAINLIT AGENTSPEC

Um agente de verdade decide sozinho. AgentSpec projeta, o agente executa.

DIA 3 · AGENTE · SEMANA AI DATA ENGINEER 2026

RECAP · DIA 2

O Que Construímos Ontem

- **Prompt Engineering → Context Engineering**
  O que você dá ao modelo define o que o modelo consegue fazer
- **LlamaIndex + Qdrant indexando reviews reais**
  Pipeline de RAG completo com dados do ShopAgent
- **MCP Supabase + MCP Qdrant conectados**
  Protocolo unificado para Ledger e Memory
- **Claude Code pesquisando nos dois stores**
  SQL exato + busca semântica — vocês decidiram qual usar

COMMUNITY · DIA 2

A Voz da Comunidade

2.452

Mensagens no Chat

DIA 2 · ZOOM CHAT

▲ Recorde absoluto

Vocês quebraram o chat do Zoom. Tema #1: "BI vai morrer?" – a noite inteira.

75

FODAs no Chat

PICO: DASHBOARD AO VIVO

▲ 15 FODAs em 2 minutos

Dashboard de Meta Ads gerado ao vivo. "LUAN ME FEZ VER QUE ESTOU PERDENDO DINHEIRO" – João Amazonas

220+

Participantes Ativos

NOMES ÚNICOS NO CHAT

▲ Debate real e ao vivo

Ferramenta #1 da comunidade: Claude. Seguido por ChatGPT, Gemini e Perplexity.

3h53

Duração da Sessão

ATÉ 00:44 DA MANHÃ

▲ "Já estamos no Dia 3"

"O BI quer morrer mas não deixam" – Alfredo Sabocinski. Luan: "Vai existir que nem COBOL".

AGENDA DA NOITE

5 Blocos, 1 Missão

20h00
Conceito de Agentes
Tool vs Agent vs Workflow – taxonomia clara

→

20h30
Design Patterns
ReAct, Routing, Chain-of-Thought

→

21h00
AgentSpec · Modo Titan
Brainstorm → Define → Design → Build

→

22h00
LangChain + Chainlit
Agente autônomo com interface real

→

22h40
Encerramento
Recap + preview Dia 4 multi-agent

BLOCO 1 · CONCEITOS

01 O Que É um Agente

PROGRESSÃO DE AUTONOMIA

4 Noites, 4 Níveis

DIA 1

EU FAÇO, IA AJUDA

Claude Code assiste na geração de dados e validação

COPILOTO

DIA 2

IA BUSCA, EU PERGUNTO

RAG + Ledger via MCP – a IA pesquisa nos seus dados

CONTEXTO

DIA 3 HOJE

IA DECIDE, EU VALIDO

AgentSpec projeta, o agente executa. Autonomia real com LangChain + Chainlit

AUTONOMIA

DIA 4

IA CONSTRÓI, IA EXECUTA

CrewAI com 3 agentes + Frontend profissional + Cloud

AMANHÃ

TAXONOMIA

Tool vs Agent vs Workflow

Tool

Eu chamo, ele executa

Chave de fenda

*Sem decisão própria – **supabase_execute_sql(query)***

DETERMINÍSTICO | INPUT/OUTPUT

Agent

Eu pergunto, ele decide como

Robô com ferramentas

*Autonomia para escolher – **"isso é SQL ou semântica?"***

AUTÔNOMO | HOJE

Workflow

Eu defino, agentes executam

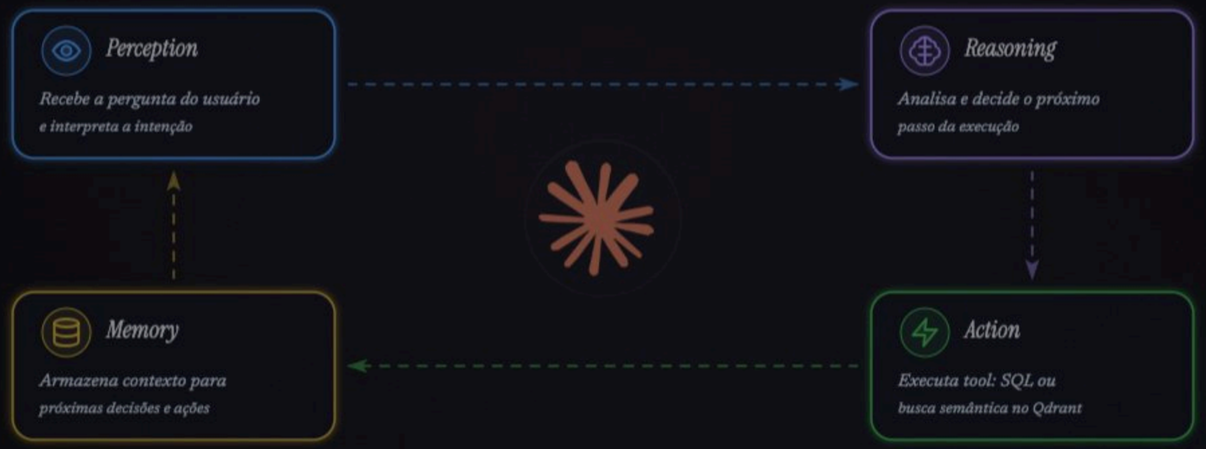
Fábrica de robôs

*Orquestração multi-agent – **CrewAI: Analyst → Researcher → Reporter***

ORQUESTRADO | DIA 4

ANATOMIA

O Loop do Agente



TIPOS DE WORKFLOW

4 Padrões de Orquestração

Sequential

A → B → C

*Cada agente executa em ordem fixa – **Analyst → Researcher → Reporter***

Parallel

A | B | C → merge

*Múltiplos agentes executam ao mesmo tempo – **resultado consolidado no final***

Hierarchical

Manager → Workers

*Um planner central delega tarefas – **specialists executam e reportam***

Consensus

A + B + C → Vote

*Agentes independentes votam – **a melhor resposta vence por consenso***

A PERGUNTA

Mas como isso funciona em produção?

Vocês acabaram de aprender 4 padrões de orquestração.
Agora imaginem alguém que usa isso **todos os dias**.

Luan Moreno
Instrutor · AIDE Brasil

MINHA VEZ

Ele mostrou em produção. Agora eu construo do zero.

LANGCHAIN

O Motor do Ecossistema

Três pacotes, um agente. Cada camada resolve um problema diferente.

CHAINLIT

A Voz do *Agente*

Chainlit transforma **30 linhas de Python** *em uma interface conversacional profissional. Sem React, sem frontend — o agente ganha* **voz e transparencia**.

chainlit_app.py — ShopAgent · UI

```
1 @cl.on_chat_start
    → agent = create_shopagent()
    → cl.user_session.set("agent", agent)

2 @cl.on_message
    → agent.astream_events(version="v2")
    → Escuta: on_chat_model_stream

3 on_tool_start
    → step = cl.Step(name="supabase_execute_sql")
    → Resultado visivel no chat ✓

4 stream_token
    → await msg.stream_token(token)
    → Resposta aparece token a token

▷ chainlit run app.py -w                    hot-reload
```

O Chainlit da **voz ao agente**. *O usuario ve o pensamento, nao so a resposta.* **Transparencia** *gera confianca.*

STREAMING · TOOL STEPS · SESSION · LIFECYCLE

TOOLS.PY

Os Blocos de *Construção*

Duas ferramentas definem o que o agente **pode fazer**. *O docstring é a especificação — o agente lê para decidir* **quando usar** *cada uma.*

```python
# src/day3/tools.py

1 @tool decorator
    → O agente descobre via docstring
    → Tipo: str → str

2 supabase_execute_sql
    → 7 queries SQL por keyword matching
    → "Faturamento, pedidos, ticket médio" → SQL ✓

3 qdrant_semantic_search
    → LlamaIndex + embeddings + top_k=5
    → "Reclamações, sentimento" → Qdrant ✓

⚠ Docstrings ruins = decisões ruins
```

7 queries SQL mapeadas por keywords + busca semântica via LlamaIndex. O agente recebe os ***resultados formatados como texto*** *— ele nunca vê SQL bruto.*

AGENT.PY + CHAINLIT_APP.PY

O Cérebro e a *Face*

*O **agent.py** cria o agente autônomo em 6 linhas. O **chainlit_app.py** dá a interface conversacional. Juntos: backend + frontend em 40 linhas.*

O Cérebro + A Face — 3 files

● AGENT.PY

```
1 create_shopagent()
    → ChatAnthropic(model="claude-sonnet-4-20250514")
    → create_react_agent(model, tools, prompt)
```

● CHAINLIT_APP.PY

```
2 @cl.on_chat_start
    → agent = create_shopagent()
    → cl.user_session.set("agent", agent)

3 @cl.on_message + stream
    → agent.astream_events(version="v2")
    → await msg.stream_token(token) ✓
```

▷ chainlit run app.py -w — 6 linhas = agente autônomo

*3 arquivos. **tools.py** dá os poderes, **agent.py** dá o cérebro, **chainlit_app.py** dá a face. Cada um com **responsabilidade única**.*

CREATE_REACT_AGENT | ASTREAM_EVENTS | CL.STEP

No Dia 2 vocês decidiram qual store usar. Hoje o agente decide sozinho.

A MUDANÇA DO DIA 3

BLOCO 2 · DESIGN PATTERNS

Como o Agente Pensa

REACT PATTERN

Reason + Act

THOUGHT

Pensamento

Preciso de dados exatos ou semânticos?

1

ACTION

Ação

Executa supabase_tool ou qdrant_tool

2

OBSERVATION

Observação

Recebe resultado da ferramenta

3

ANSWER

Resposta

Sintetiza resposta final ao usuário

4

loop até resposta completa

*ReAct é o **padrão fundamental** dos agentes modernos. Cada ciclo de pensamento-ação-observação **refina a resposta** progressivamente.*

REACT REASON+ACT LANGCHAIN

REACT · EXEMPLO REAL

O Agente *Raciocina*

Uma pergunta complexa aciona **duas ferramentas** *em sequência. O agente decide sozinho quando usar* **busca semântica** *e quando usar* **SQL exato**.

Claude Code ShopAgent · ReAct

> *"Clientes premium do Sudeste que reclamam de entrega: ticket médio?"*

Thought: Preciso encontrar quem reclama (semântico)
Action: qdrant_semantic_search("reclamação entrega sudeste")
Observe: → 23 reviews (score > 0.82)

Thought: Agora filtrar Sudeste e calcular média
Action: supabase_execute_sql("SELECT AVG(total)…")
Observe: → avg_total: **347.82**

Answer: "R$ 347,82 — 23 reviews negativos do Sudeste"

● 2 tools · 2 cycles · dual-store response

O agente usou ***ambas as pernas*** *em uma única pergunta. Semântica para encontrar, SQL para calcular –* ***autonomia real***.

DUAL-STORE AUTONOMIA REACT

TOOL USE

O Agente Escolhe

THE LEDGER

Isso é SQL

*Dados exatos: **números, contagens, médias, JOINs**. Sempre que a pergunta pede um **valor preciso**, o agente gera SQL e consulta o Postgres.*

- *"Faturamento total de SP?"*
- *"Top 5 produtos por receita?"*
- *"Ticket médio por segmento?"*
- *"Pedidos por estado este mês?"*

`supabase_execute_sql`

THE MEMORY

Isso é Semântica

*Significado: **reclamações, sentimento, temas de review**. Quando a pergunta pede **contexto humano**, o agente busca vetores no Qdrant.*

- *"Quem reclama de entrega?"*
- *"Sentimento sobre marca X?"*
- *"Temas negativos do mês?"*
- *"O que elogiam na categoria?"*

`qdrant_semantic_search`

*O agente **decide sozinho** qual ferramenta usar. Uma pergunta, **duas pernas**, resposta completa.*

LEDGER MEMORY ROUTING

ROUTING

Como o Agente *Decide*

*O routing é o **coração da inteligência** do agente. Classificação em linguagem natural **decide o caminho** antes de qualquer execução.*

NL ROUTING LEDGER MEMORY

CHAIN-OF-THOUGHT

Pensamento *Visível*

O agente mostra **cada passo** *do raciocínio antes de agir. Cada decisão é* **auditável** *– essencial para confiança em produção.*

```
Claude Code                                          ShopAgent · CoT

> "Clientes premium do Sudeste que reclamam de entrega: ticket médio?"

1 Classificar a pergunta
    → sentimento → semântico ✓
    → números → SQL ✓
    → Decisão: HÍBRIDO

2 Busca semântica
    → Qdrant: "insatisfação entrega"
    → 23 reviews, score 0.87

3 Query SQL
    → SELECT AVG(total) FROM orders
      WHERE state IN ('SP','RJ','MG','ES')
    → R$ 347,82

4 Síntese
    → dados + contexto → resposta final

● 4 etapas · raciocínio transparente · auditável
```

Chain-of-Thought torna o ***raciocínio transparente*** *– cada passo é auditável. Isso é essencial para construir* ***confiança em produção*** *com agentes autônomos.*

COT RACIOCÍNIO TRANSPARÊNCIA

RESUMO

4 Padrões, 1 Agente

ReAct
O loop fundamental

Quando: ***toda pergunta*** *passa por Think-Act-Observe.*

NA PRÁTICA:

ShopAgent raciocina se precisa do Ledger ou da Memory antes de agir.

Tool Use
Escolha de ferramenta

Quando: ***múltiplas fontes*** *de dados disponíveis.*

NA PRÁTICA:

Agente escolhe supabase_tool para "faturamento" e qdrant_tool para "sentimento".

Routing
Classificação inteligente

Quando: ***SQL vs semântica*** *precisa ser decidido.*

NA PRÁTICA:

NL classifier detecta "reclamação" e roteia para Qdrant automaticamente.

Chain-of-Thought
Raciocínio explícito

Quando: ***decisões complexas*** *precisam de auditoria.*

NA PRÁTICA:

Cada passo do ShopAgent é logado — auditável e reproduzível.

Estes 4 padrões ***combinados*** *formam a inteligência do ShopAgent. Cada padrão é uma* ***camada de decisão****.*

REACT TOOL USE ROUTING COT

SPEC-DRIVEN DEVELOPMENT

Spec-Driven Development

THE EVOLUTION

Three Eras of *Control*

Control freaks aren't losing control. They're **scaling it**.

THE REFRAME

You Don't Control the Steps.
You Control the Guardrails.

LLMs are **non-deterministic** *by design — the same prompt produces different token sequences every time. But the* **output quality** *can be made deterministic when you constrain the* **verification space**, *not the generation space.*

Vague	High variance — hallucinations
Spec	Constrained
Spec+Tests	Deterministic outcome

THE CONVERGENCE FORMULA

∞ (Code Paths) × 7 (Gates) = 1 (Outcome)

You don't control how the code is written.
You control **whether it ships**.

Under the Hood — Why This Works

1 *Context = Constraint*
CLAUDE.md e KB specs ocupam a **context window**. *Mais spec =* **distribuição mais estreita** *sobre os próximos tokens.*

2 *Test Loop = Convergência*
O agente executa **write → test → read error → fix → re-test** *em loop. Após 2-3 iterações, todas as soluções convergem para o* **mesmo comportamento observável**.

3 *Gates = Fronteira Rígida*
Quality gates atuam como **filtro binário accept/reject**. *A variância no código é irrelevante porque a variância no resultado é zero.*

- *Processo não-determinístico × verificação determinística =* **resultado determinístico**.

What Is Spec-Driven Development?

THE BACKBONE OF SDD

Product Requirements Document

No SDD, um PRD não é um documento passivo no Confluence. É uma **especificação legível por máquina** *que agentes de IA consomem, validam e transformam em código. A spec é o produto.*

DEFINE_ShopAgent.md ● 14/15

01 *Problem Statement*
Qual problema de negócio o ShopAgent resolve?

02 *Data Architecture*
The Ledger (Postgres) + The Memory (Qdrant) — dual-store

03 *Tool Definitions*
supabase_execute_sql, qdrant_semantic_search — inputs/outputs

04 *Edge Cases & Routing*
Queries ambíguas, fallback, multi-tool chains

05 *Acceptance Tests*
Given/When/Then — o que precisa passar antes do deploy

06 *Data Contract*
Pydantic models, response schemas, Chainlit interface

CLARITY SCORE | Problem 3/3 | Users 3/3 | Goals 3/3 | Success 2/3 | Scope 3/3 | 14/15 ✓

"The specification becomes the source of truth and determines what gets built."
— GitHub Engineering, September 2025

O PRD é a **espinha dorsal** *do SDD — agentes consomem, validam e transformam a spec em código* **automaticamente**.

PRD | CLARITY SCORE | MACHINE-READABLE

SDD LANDSCAPE

Três Frameworks, Uma Revolução

SDD nasceu em 2025 com a onda de AI coding agents. Três frameworks lideram o movimento — cada um com uma filosofia diferente.

WHY AGENTSPEC?

O Que Ninguém Mais Faz

SpecKit e OpenSpec dão **estrutura** *para specs. AgentSpec dá* **inteligência de domínio**. *A diferença? Um agente genérico interpreta sua spec.* **58 especialistas** *executam com padrões validados e quality gates quantificados.*

Comparison Matrix

DIMENSION	SpecKit	OpenSpec	AgentSpec
Approach	Horizontal	Horizontal	**Vertical**
Specialist Agents	0	0	**58**
KB Domains	0	0	**23**
Slash Commands	~4	9+	**29**
Quality Gates	Verify	Pass/Fail	**Scored/Phase**
Domain Focus	General	General	**Data Eng**
Institutional Memory	✗	✗	**42.5K lines**

SpecKit e OpenSpec dizem **"escreva a spec"**.
AgentSpec diz **"escreva a spec, e 58 especialistas vão construir com a sabedoria de 42.500 linhas de padrões validados"**.

Horizontal é necessário. **Vertical é transformador.** *AgentSpec combina SDD com multi-agent delegation para data engineering em escala.*

VERTICAL | 58 AGENTS | KB-FIRST

THE FRAMEWORK

AgentSpec

A revolutionary **Agentic Operating System** *for data engineering. 58 specialist agents backed by 42,500 lines of institutional memory.* **No AI session starts from scratch again.** *Every pattern, every decision, every lesson learned — preserved across sessions, teams, and projects.*

Agentic OS

INTELLIGENCE STACK

UI	**Slash Commands** *29 commands — /brainstorm, /define, /design, /build, /ship*	29
WF	**Workflow Engine** *5-phase SDD — Brainstorm → Define → Design → Build → Ship*	5
AG	**Agent Tier System** *58 specialists — 10 utility, 28 domain experts, 20 platform*	58
KB	**Knowledge Base** *22 domains — 311 files — 42,500+ lines of validated patterns*	42.5K
MV	**Validation Engine** *KB-first → MCP fallback → Agreement Matrix → confidence*	MCP

58 agentes, 22 domínios de conhecimento, 5 fases — **memória institucional** *que nunca se perde.*

AGENTSPEC CLAUDE CODE v2.0

WHAT IS AGENTSPEC?

Your AI Team's *Operating System*

AgentSpec is a **Claude Code plugin** *that gives AI agents institutional memory. Instead of starting from scratch every session, agents inherit* **289 knowledge base files**, **29 slash commands**, *and a battle-tested* **5-phase workflow**.

github.com/**luanmorenommaciel**/agentspec MIT ★ 68 ⚲ 44

Um plugin que transforma cada sessão de IA em **memória institucional** *– 42.500+ linhas de padrões validados.*

INSTITUTIONAL MEMORY | KB-FIRST | 22 DOMAINS

THE AGENT ROSTER

58 Agents · 22 KB Domains · 3 *Tiers*

Architect — System design, cloud infra	12
Data Eng — Spark, dbt, Lakeflow	14
Python — Parsers, Lambda, clean code	10
Workflow — SDD 5-phase lifecycle	8
Cloud — AWS, GCP, Databricks	6
Test — pytest, fixtures, CI gates	4
Platform — Fabric, Snowflake	2
Dev — Shell, CI/CD, review	2

10	**28**	**20**
T1 UTILITY	T2 DOMAIN	T3 PLATFORM
Shell, review, docs	Spark, Lambda, TSYS	Medallion, GenAI

AGENT ROUTING — HOW TASKS GET MATCHED

User Prompt (/build parser) → *Keyword Scan* (file → agent index) → *KB Lookup* (load domain context) → *Execute* (specialist agent)

>_ 29 Slash Commands

6 GROUPS

/brainstorm · /define · /design · /build · /ship · /iterate · /review · /validate · +21 more

Knowledge Bases

311 files · 42,500+ lines

54 **Fabric** — Microsoft Fabric, lakehouses, notebooks	23 **Lakeflow** — DLT pipelines, streaming tables, DABs
22 **AWS** — Lambda, S3, SAM, CloudFormation	15 **Terraform** — IaC modules, state, workspaces
14 **GCP** — BigQuery, Dataflow, Cloud Run	12 **GenAI** — LLM patterns, RAG, agents
12 **dbt** — Transforms, tests, macros	12 **Prompts** — Extraction, chain-of-thought
11 **Spark** — RDDs, DataFrames, tuning	11 **Medallion** — Bronze, Silver, Gold layers

+12 more domains:

Airflow · Testing · Pydantic · Python · Streaming · SQL · Data Modeling · Data Quality · Lakehouse · Modern Stack · Cloud · AI Pipelines

THE 5-PHASE WORKFLOW

Five Phases to *Production*

Each phase produces a versioned artifact. Quality gates enforce minimum scores. Agents auto-delegate based on file manifests. **No phase is skipped.**

DEMO AO VIVO

Vamos Implementar o AgentSpec

INSTALAÇÃO

Instale o AgentSpec

Abram o terminal agora. Um comando e vocês têm 58 agentes no sistema.

PHASE 1 OF 5

/brainstorm
Discover

Before writing code, the agent asks **6 discovery questions** *to build a mental model of the ShopAgent. It evaluates architectures and proposes approaches with* **confidence scores**.

● Claude Code Opus 4.6

```
> /brainstorm ShopAgent — agente e-commerce com Qdrant + Supabase, LangChain ReAct,
Chainlit streaming
```

Output: spec/01-brainstorm.md

```
# Approaches Proposed
A. LangChain ReAct Agent [0.92]
B. LangGraph StateGraph [0.85]
C. CrewAI single-agent [0.78]

Recommendation: A (ReAct + dual-store)
```

● **Gate: Can describe the full agent loop end-to-end**

PHASE 2 OF 5

/define
Extract

The agent extracts **structured requirements**, *maps functional needs with MoSCoW priorities, writes acceptance tests, and produces a* **clarity score ≥ 12/15**.

● Claude Code Opus 4.6

```
> /define SHOPAGENT — extract requirements, acceptance tests, clarity score
```

Output: spec/02-define.md

```
# Acceptance Tests
AT-01 → "Faturamento?" → SQL tool
AT-02 → "Reclamacoes?" → Qdrant tool
AT-03 → "Premium + reclamam?" → BOTH
AT-04 → Chainlit tool steps → visible

# Clarity Score: 14/15
Problem 3/3 · Users 3/3 · Goals 3/3
Success 2/3 · Scope 3/3
```

● Gate: Clarity score 14/15 ≥ 12 — APPROVED

PHASE 3 OF 5

/design *Architect*

The agent produces a **5-file manifest**, *assigns each to a specialist agent, defines the ReAct pipeline architecture, and maps the full Chainlit streaming flow.*

● Claude Code Opus 4.6

```
> /design SHOPAGENT — file manifest, agent assignment, pipeline architecture
```

Output: spec/03-design.md

```
# File Manifest + Agent Delegation
src/day3/tools.py → python-developer
src/day3/agent.py → genai-architect
src/day3/chainlit_app.py → python-developer

# Pipeline Architecture
User → Chainlit → ReAct → [Ledger | Memory]
```

● Gate: Design reviewed — manifest complete — APPROVED

PHASE 4 OF 5

/build
Implement

The agent reads the DESIGN manifest, **auto-delegates each file to its specialist**, *writes production code with streaming, and produces a BUILD report with* **zero errors**.

● Claude Code Opus 4.6

```
> /build SHOPAGENT — execute manifest, delegate to specialists, test routing
```

<> *Output: Code + Tests + Report*

```
✶ Executing DESIGN manifest…
python-developer → tools.py ✓ 7 SQL queries
genai-architect → agent.py ✓ ReAct + streaming
python-developer → chainlit_app.py ✓ cl.Step
code-reviewer → review ✓ 0 issues

3/3 routing tests passed · ~200 LOC
```

● Gate: All tests pass, 3 agents delegated, 0 errors

PHASE 5 OF 5

/ship
Deploy

The agent verifies **end-to-end functionality**, *documents the architecture, archives learnings to the knowledge base, and prepares the* **Day 4 multi-agent handoff**.

● Claude Code Opus 4.6

```
> /ship SHOPAGENT — verify e2e, archive learnings, prepare Day 4 handoff
```

→ *Output: Production Verified*

```
$ docker compose ps
postgres:16 healthy ✓ · qdrant:v1.13 healthy ✓

$ python3 src/day3/agent.py
3/3 routing ✓ · Chainlit streaming ✓

# Day 4 Handoff
1 agent → 3 agents (CrewAI)
sequential → parallel execution
```

● Gate: E2E verified, learnings archived, Day 4 ready

5 FASES COMPLETAS

Do brainstorm ao deploy. Isso é o que construímos.

2
Tools
supabase_execute_sql
qdrant_semantic_search

1
Agente ReAct
Decide sozinho qual store consultar

1
Chat Interface
Chainlit com streaming e tool steps visíveis

ARQUITETURA COMPLETA

O Sistema do Dia 3

User → Chainlit → Agent → [Ledger | Memory] → Response

BLOCO 5 · ENCERRAMENTO

O Que Vocês Construíram

05

PREVIEW · DIA 4

Amanhã: O Grand Finale

CrewAI

Analyst → Researcher → Reporter

*3 agentes especializados colaboram em um **relatório executivo completo***

DeepEval

assert score > 0.85

*Testes automatizados validando **cada decisão do agente***

LangFuse

trace → observe → optimize

*Observabilidade de cada chamada LLM – **custo, latência, qualidade***

Frontend Pro

Dashboard + Chainlit embeddable

*Interface profissional com **chat embeddable e dashboard visual***

“O agente já decide sozinho. Amanhã, ele ganha um time.”

*Vocês saem desta noite com o **conceito de agentes dominado**, o **AgentSpec configurado**, e um **agente LangChain com interface Chainlit** funcionando. Amanhã, o grand finale: multi-agent, avaliação, frontend profissional, e cloud.*